# Astra Control Center를 설치합니다

Astra Control Center

NetApp
March 27, 2024

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-center/get-started/cert-manager-prereqs.html on March 27, 2024. Always check docs.netapp.com for the latest.

# 목차

표준 프로세스를 사용하여 Astra Control Center를 설치합니다 . . . 1
Astra Control Center를 다운로드하고 압축을 풉니다 . . . 3
로컬 레지스트리를 사용하는 경우 추가 단계를 완료합니다 . . . 4
인증 요구 사항이 있는 레지스트리에 대한 네임스페이스 및 암호를 설정합니다 . . . 8
Astra Control Center 운영자를 설치합니다 . . . 9
Astra Control Center를 구성합니다 . . . 12
Astra 제어 센터 및 운전자 설치를 완료합니다 . . . 25
시스템 상태를 확인합니다 . . . 26
부하 분산을 위한 수신 설정 . . . 33
Astra Control Center UI에 로그인합니다 . . . 36
설치 문제를 해결합니다 . . . 37
대체 설치 절차 . . . 37
다음 단계 . . . 37
외부 인증서 관리자를 구성합니다 . . . 37

# 표준 프로세스를 사용하여 **Astra Control Center**를 설치합니다

Astra Control Center를 설치하려면 설치 이미지를 다운로드하고 다음 단계를 수행하십시오. 이 절차를 사용하여 인터넷에 연결되었거나 공기가 연결된 환경에 Astra Control Center를 설치할 수 있습니다.

Astra Control Center 설치 프로세스 데모는 를 참조하십시오 "이 비디오".

시작하기 전에

- * 환경 필수 조건 충족 *: "설치를 시작하기 전에 Astra Control Center 구축을 위한 환경을 준비합니다".

> Astra Control Center를 세 번째 고장 도메인 또는 보조 사이트에 배포합니다. 앱 복제 및 원활한 재해 복구에 권장됩니다.

- * 건강한 서비스 보장 * : 모든 API 서비스가 정상 상태이고 사용 가능한지 확인하십시오.

```
kubectl get apiservices
```

- * 라우팅 가능한 FQDN *: 사용하려는 Astra FQDN을 클러스터로 라우팅할 수 있습니다. 즉, 내부 DNS 서버에 DNS 항목이 있거나 이미 등록된 코어 URL 경로를 사용하고 있는 것입니다.
- * 인증서 관리자 구성 *: 클러스터에 이미 인증서 관리자가 있는 경우 일부 작업을 수행해야 합니다 "필수 단계" 따라서 Astra Control Center는 자체 인증 관리자를 설치하려고 시도하지 않습니다. 기본적으로 Astra Control Center는 설치 중에 자체 인증서 관리자를 설치합니다.
- * (ONTAP SAN 드라이버만 해당) 다중 경로 사용 *: ONTAP SAN 드라이버를 사용하는 경우 모든 Kubernetes 클러스터에서 다중 경로가 활성화되어 있는지 확인하십시오.

다음 사항도 고려해야 합니다.

- * NetApp Astra Control 이미지 레지스트리에 액세스 *:

  NetApp 이미지 레지스트리에서 Astra Control Provisioner와 같은 Astra Control의 설치 이미지 및 기능 개선 사항을 가져올 수 있습니다.

  a. 레지스트리에 로그인해야 하는 Astra Control 계정 ID를 기록합니다.

     계정 ID는 Astra Control Service 웹 UI에서 확인할 수 있습니다. 페이지 오른쪽 상단의 그림 아이콘을 선택하고 * API 액세스 * 를 선택한 후 계정 ID를 기록합니다.

  b. 같은 페이지에서 * API 토큰 생성 * 을 선택하고 API 토큰 문자열을 클립보드에 복사하여 편집기에 저장합니다.

  c. Astra Control 레지스트리에 로그인합니다.

```
docker login cr.astra.netapp.io -u <account-id> -p <api-token>
```

- * 보안 통신을 위한 서비스 메시를 설치합니다 * : Astra Control 호스트 클러스터 통신 채널은 을 사용하여 보안을

유지하는 것이 좋습니다 "지원되는 서비스 메시입니다".

> Astra Control Center를 서비스 메시와 통합하는 작업은 Astra Control Center 중에만 수행할 수 있습니다 "설치" 그리고 이 과정에 독립적이지 않습니다. 메시에서 메시되지 않은 환경으로 다시 변경하는 것은 지원되지 않습니다.

Istio 서비스 메시를 사용하려면 다음을 수행해야 합니다.

- 를 추가합니다 `istio-injection:enabled` 라벨 Astra Control Center를 구축하기 전에 Astra 네임스페이스에 매핑
- 를 사용합니다 `Generic` 수신 설정 에 대한 대체 침입을 제공합니다 외부 부하 균형.
- Red Hat OpenShift 클러스터의 경우 을 정의해야 합니다 `NetworkAttachmentDefinition` 연결된 모든 Astra Control Center 네임스페이스에서 (`netapp-acc-operator`, `netapp-acc`, `netapp-monitoring` 응용 프로그램 클러스터 또는 대체된 사용자 지정 네임스페이스의 경우).

```
cat <<EOF | oc -n netapp-acc-operator create -f -
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: istio-cni
EOF

cat <<EOF | oc -n netapp-acc create -f -
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: istio-cni
EOF

cat <<EOF | oc -n netapp-monitoring create -f -
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: istio-cni
EOF
```

단계

Astra Control Center를 설치하려면 다음 단계를 수행하십시오.

- Astra Control Center를 다운로드하고 압축을 풉니다
- 로컬 레지스트리를 사용하는 경우 추가 단계를 완료합니다
- 인증 요구 사항이 있는 레지스트리에 대한 네임스페이스 및 암호를 설정합니다
- Astra Control Center 운영자를 설치합니다
- Astra Control Center를 구성합니다

- Astra 제어 센터 및 운전자 설치를 완료합니다
- 시스템 상태를 확인합니다
- 부하 분산을 위한 수신 설정
- Astra Control Center UI에 로그인합니다

> Astra Control Center 운영자를 삭제하지 마십시오(예: `kubectl delete -f astra_control_center_operator_deploy.yaml`) 포드가 삭제되지 않도록 Astra Control Center 설치 또는 작동 중에 언제든지.

# Astra Control Center를 다운로드하고 압축을 풉니다

다음 위치 중 하나에서 Astra Control Center 이미지를 다운로드하십시오.

- * Astra 컨트롤 서비스 이미지 레지스트리 *: Astra 컨트롤 센터 이미지에 로컬 레지스트리를 사용하지 않거나 NetApp Support 사이트에서 번들 다운로드보다 이 방법을 선호하는 경우 이 옵션을 사용합니다.
- * NetApp Support 사이트 *: Astra 컨트롤 센터 이미지와 함께 로컬 레지스트리를 사용하는 경우 이 옵션을 사용합니다.

**Astra Control** 이미지 레지스트리

1. Astra Control Service에 로그인합니다.
2. 대시보드에서 * Astra Control의 자가 관리형 인스턴스 배포 * 를 선택합니다.
3. 지침에 따라 Astra Control 이미지 레지스트리에 로그인하고 Astra Control Center 설치 이미지를 가져온 다음 이미지를 추출합니다.

**NetApp Support** 사이트

1. Astra Control Center가 포함된 번들을 다운로드합니다 (`astra-control-center-[version].tar.gz`)를 선택합니다 "Astra Control Center 다운로드 페이지".
2. (권장되지만 선택 사항) Astra Control Center용 인증서 및 서명 번들을 다운로드합니다 (`astra-control-center-certs-[version].tar.gz`)를 클릭하여 번들 서명을 확인합니다.

```
tar -vxzf astra-control-center-certs-[version].tar.gz
```

```
openssl dgst -sha256 -verify certs/AstraControlCenter-public.pub
-signature certs/astra-control-center-[version].tar.gz.sig astra-
control-center-[version].tar.gz
```

   출력이 표시됩니다 `Verified OK` 확인 성공 후.

3. Astra Control Center 번들에서 이미지를 추출합니다.

```
tar -vxzf astra-control-center-[version].tar.gz
```

# 로컬 레지스트리를 사용하는 경우 추가 단계를 완료합니다

Astra Control Center 번들을 로컬 레지스트리에 푸시하려는 경우 NetApp Astra kubectl 명령줄 플러그인을 사용해야 합니다.

## NetApp Astra kubtl 플러그인을 설치합니다

최신 NetApp Astra kubectl 명령줄 플러그인을 설치하려면 다음 단계를 완료하십시오.

시작하기 전에

NetApp은 다양한 CPU 아키텍처 및 운영 체제에 대한 플러그인 바이너리를 제공합니다. 이 작업을 수행하기 전에 사용 중인 CPU 및 운영 체제를 알아야 합니다.

이전 설치에서 이미 플러그인을 설치한 경우 "최신 버전이 있는지 확인하십시오" 다음 단계를 수행하기 전에

단계

1. 사용 가능한 NetApp Astra kubeck 플러그인 바이너리를 나열합니다.

kubbeck 플러그인 라이브러리는 tar 번들의 일부이며 폴더에 압축이 풀립니다 `kubectl-astra`.

```
ls kubectl-astra/
```

2. 운영 체제 및 CPU 아키텍처에 필요한 파일을 현재 경로로 이동하고 이름을 로 변경합니다 `kubectl-astra`:

```
cp kubectl-astra/<binary-name> /usr/local/bin/kubectl-astra
```

## 레지스트리에 이미지를 추가합니다

1. Astra Control Center 번들을 로컬 레지스트리로 푸시하려는 경우 컨테이너 엔진에 적합한 단계 시퀀스를 완료합니다.

**Docker** 를 참조하십시오

a. 타볼의 루트 디렉토리로 변경합니다. 가 표시됩니다 `acc.manifest.bundle.yaml` 파일 및 다음 디렉토리:

```
acc/
kubectl-astra/
acc.manifest.bundle.yaml
```

b. Astra Control Center 이미지 디렉토리의 패키지 이미지를 로컬 레지스트리에 밀어 넣습니다. 를 실행하기 전에 다음 대체 작업을 수행합니다 `push-images` 명령:

- <BUNDLE_FILE>를 Astra Control 번들 파일의 이름으로 바꿉니다 (`acc.manifest.bundle.yaml`)를 클릭합니다.
- <MY_FULL_REGISTRY_PATH>를 Docker 저장소의 URL로 바꿉니다. 예를 들어, "<a href="https://<docker-registry>"" class="bare">https://<docker-registry>"</a>.
- <MY_REGISTRY_USER>를 사용자 이름으로 바꿉니다.
- <MY_REGISTRY_TOKEN>를 레지스트리에 대한 인증된 토큰으로 바꿉니다.

```
kubectl astra packages push-images -m <BUNDLE_FILE> -r
<MY_FULL_REGISTRY_PATH> -u <MY_REGISTRY_USER> -p
<MY_REGISTRY_TOKEN>
```

팟맨

a. 타볼의 루트 디렉토리로 변경합니다. 이 파일과 디렉토리가 표시됩니다.

```
acc/
kubectl-astra/
acc.manifest.bundle.yaml
```

b. 레지스트리에 로그인합니다.

```
podman login <YOUR_REGISTRY>
```

c. 사용하는 Podman 버전에 맞게 사용자 지정된 다음 스크립트 중 하나를 준비하고 실행합니다. <MY_FULL_REGISTRY_PATH>를 모든 하위 디렉토리가 포함된 리포지토리의 URL로 대체합니다.

```
<strong>Podman 4</strong>
```

```
export REGISTRY=<MY_FULL_REGISTRY_PATH>
export PACKAGENAME=acc
export PACKAGEVERSION=24.02.0-69
export DIRECTORYNAME=acc
for astraImageFile in $(ls ${DIRECTORYNAME}/images/*.tar) ; do
astraImage=$(podman load --input ${astraImageFile} | sed
's/Loaded image: //')
astraImageNoPath=$(echo ${astraImage} | sed 's:.*/::')
podman tag ${astraImageNoPath} ${REGISTRY}/netapp/astra/
${PACKAGENAME}/${PACKAGEVERSION}/${astraImageNoPath}
podman push ${REGISTRY}/netapp/astra/${PACKAGENAME}/
${PACKAGEVERSION}/${astraImageNoPath}
done
```

```
<strong>Podman 3</strong>
```

```
export REGISTRY=<MY_FULL_REGISTRY_PATH>
export PACKAGENAME=acc
export PACKAGEVERSION=24.02.0-69
export DIRECTORYNAME=acc
for astraImageFile in $(ls ${DIRECTORYNAME}/images/*.tar) ; do
astraImage=$(podman load --input ${astraImageFile} | sed
's/Loaded image: //')
astraImageNoPath=$(echo ${astraImage} | sed 's:.*/::')
podman tag ${astraImageNoPath} ${REGISTRY}/netapp/astra/
${PACKAGENAME}/${PACKAGEVERSION}/${astraImageNoPath}
podman push ${REGISTRY}/netapp/astra/${PACKAGENAME}/
${PACKAGEVERSION}/${astraImageNoPath}
done
```

레지스트리 구성에 따라 스크립트가 만드는 이미지 경로는 다음과 같아야 합니다.

```
https://downloads.example.io/docker-astra-control-
prod/netapp/astra/acc/24.02.0-69/image:version
```

2. 디렉토리를 변경합니다.

```
cd manifests
```

# 인증 요구 사항이 있는 레지스트리에 대한 네임스페이스 및 암호를 설정합니다

1. Astra Control Center 호스트 클러스터의 kubecononfig 내보내기:

```
export KUBECONFIG=[file path]
```

> 설치를 완료하기 전에 Astra Control Center를 설치할 클러스터를 추천하십시오.

2. 인증이 필요한 레지스트리를 사용하는 경우 다음을 수행해야 합니다.

   a. 'NetApp-acc-operator' 네임스페이스 생성:

```
kubectl create ns netapp-acc-operator
```

   b. NetApp-acc-operator 네임스페이스에 대한 암호를 생성합니다. Docker 정보를 추가하고 다음 명령을 실행합니다.

> 자리 표시자입니다 `your_registry_path` 이전에 업로드한 이미지의 위치와 일치해야 합니다(예: `[Registry_URL]/netapp/astra/astracc/24.02.0-69`)를 클릭합니다.

```
kubectl create secret docker-registry astra-registry-cred -n netapp-acc-
operator --docker-server=cr.astra.netapp.io --docker
-username=[astra_account_id] --docker-password=[astra_api_token]
```

+

```
kubectl create secret docker-registry astra-registry-cred -n netapp-acc-
operator --docker-server=[your_registry_path] --docker
-username=[username] --docker-password=[token]
```

+

> 암호를 생성한 후 네임스페이스를 삭제하면 네임스페이스를 다시 만든 다음 네임스페이스에 대한 암호를 다시 생성합니다.

   a. 를 생성합니다 `netapp-acc` (또는 사용자 지정 이름) 네임스페이스입니다.

```
kubectl create ns [netapp-acc or custom namespace]
```

   b. 에 대한 암호를 만듭니다 `netapp-acc` (또는 사용자 지정 이름) 네임스페이스입니다. Docker 정보를 추가하고

레지스트리 기본 설정에 따라 적절한 명령 중 하나를 실행합니다.

```
kubectl create secret docker-registry astra-registry-cred -n [netapp-acc or custom namespace] --docker-server=cr.astra.netapp.io --docker-username=[astra_account_id] --docker-password=[astra_api_token]
```

```
kubectl create secret docker-registry astra-registry-cred -n [netapp-acc or custom namespace] --docker-server=[your_registry_path] --docker-username=[username] --docker-password=[token]
```

# Astra Control Center 운영자를 설치합니다

1. (로컬 레지스트리만 해당) 로컬 레지스트리를 사용하는 경우 다음 단계를 수행하십시오.

   a. Astra Control Center 운영자 배포 YAML을 엽니다.

   ```
   vim astra_control_center_operator_deploy.yaml
   ```

   ⓘ YAML 주석이 붙은 샘플은 다음 단계를 따릅니다.

   b. 인증이 필요한 레지스트리를 사용하는 경우 'imagePullSecrets:[]'의 기본 줄을 다음과 같이 바꿉니다.

   ```
   imagePullSecrets: [{name: astra-registry-cred}]
   ```

   c. 변경 `ASTRA_IMAGE_REGISTRY` 의 경우 `kube-rbac-proxy` 이미지를 에서 푸시한 레지스트리 경로로 이미지 이전 단계.

   d. 변경 `ASTRA_IMAGE_REGISTRY` 의 경우 `acc-operator-controller-manager` 이미지를 에서 푸시한 레지스트리 경로로 이미지 이전 단계.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    control-plane: controller-manager
  name: acc-operator-controller-manager
  namespace: netapp-acc-operator
spec:
  replicas: 1
  selector:
    matchLabels:
      control-plane: controller-manager
```

```
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        control-plane: controller-manager
    spec:
      containers:
      - args:
        - --secure-listen-address=0.0.0.0:8443
        - --upstream=http://127.0.0.1:8080/
        - --logtostderr=true
        - --v=10
        image: ASTRA_IMAGE_REGISTRY/kube-rbac-proxy:v4.8.0
        name: kube-rbac-proxy
        ports:
        - containerPort: 8443
          name: https
      - args:
        - --health-probe-bind-address=:8081
        - --metrics-bind-address=127.0.0.1:8080
        - --leader-elect
        env:
        - name: ACCOP_LOG_LEVEL
          value: "2"
        - name: ACCOP_HELM_INSTALLTIMEOUT
          value: 5m
        image: ASTRA_IMAGE_REGISTRY/acc-operator:24.02.68
        imagePullPolicy: IfNotPresent
        livenessProbe:
          httpGet:
            path: /healthz
            port: 8081
          initialDelaySeconds: 15
          periodSeconds: 20
        name: manager
        readinessProbe:
          httpGet:
            path: /readyz
            port: 8081
          initialDelaySeconds: 5
          periodSeconds: 10
        resources:
          limits:
            cpu: 300m
            memory: 750Mi
```

```
          requests:
            cpu: 100m
            memory: 75Mi
        securityContext:
          allowPrivilegeEscalation: false
      imagePullSecrets: []
      securityContext:
        runAsUser: 65532
      terminationGracePeriodSeconds: 10
```

2. Astra Control Center 운영자를 설치합니다.

```
kubectl apply -f astra_control_center_operator_deploy.yaml
```

샘플 응답을 위해 확장:

```
namespace/netapp-acc-operator created
customresourcedefinition.apiextensions.k8s.io/astracontrolcenters.as
tra.netapp.io created
role.rbac.authorization.k8s.io/acc-operator-leader-election-role
created
clusterrole.rbac.authorization.k8s.io/acc-operator-manager-role
created
clusterrole.rbac.authorization.k8s.io/acc-operator-metrics-reader
created
clusterrole.rbac.authorization.k8s.io/acc-operator-proxy-role
created
rolebinding.rbac.authorization.k8s.io/acc-operator-leader-election-
rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/acc-operator-manager-
rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/acc-operator-proxy-
rolebinding created
configmap/acc-operator-manager-config created
service/acc-operator-controller-manager-metrics-service created
deployment.apps/acc-operator-controller-manager created
```

3. Pod가 실행 중인지 확인합니다.

```
kubectl get pods -n netapp-acc-operator
```

# Astra Control Center를 구성합니다

1. Astra Control Center 사용자 정의 리소스(CR) 파일을 편집합니다 (`astra_control_center.yaml`) 계정, 지원, 레지스트리 및 기타 필요한 구성을 만들려면:

```
vim astra_control_center.yaml
```

> YAML 주석이 붙은 샘플은 다음 단계를 따릅니다.

2. 다음 설정을 수정하거나 확인합니다.

계정 이름

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `accountName` | 를 변경합니다 `accountName` Astra Control Center 계정과 연결할 이름에 대한 문자열입니다. 하나의 accountName만 있을 수 있습니다. | 문자열 | `Example` |

**astraVersion**을 참조하십시오

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `astraVersion` | 배포할 Astra Control Center의 버전입니다. 값이 미리 채워질 수 있으므로 이 설정에 대한 작업은 필요하지 않습니다. | 문자열 | `24.02.0-69` |

**astraAddress**를 선택합니다

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `astraAddress` | 를 변경합니다 `astraAddress` 브라우저에서 Astra Control Center에 액세스하기 위해 사용할 FQDN(권장) 또는 IP 주소에 대한 문자열입니다. 이 주소는 Astra Control Center가 데이터 센터에서 어떻게 검색되는지 정의하며, 이 주소를 완료하면 로드 밸런서에서 제공한 것과 동일한 FQDN 또는 IP 주소입니다 "Astra Control Center 요구 사항".<br><br>참고: 사용하지 마십시오 `http://` 또는 `https://` 를 입력합니다. 에서 사용하기 위해 이 FQDN을 복사합니다 나중에. | 문자열 | `astra.example.com` |

**AutoSupport**

이 섹션에서는 NetApp의 사전 지원 애플리케이션인 NetApp Active IQ에 참여할지 여부와 데이터 전송 위치에 대해 결정합니다. 인터넷 연결이 필요하며(포트 442) 모든 지원 데이터가 익명화됩니다.

| 설정 | 사용 | 지침 | 유형 | 예 |
|---|---|---|---|---|
| `autoSupport.enrolled` | 둘 다 가능합니다 `enrolled` 또는 `url` 필드를 선택해야 합니다 | 변경 `enrolled` 을 눌러 AutoSupport to로 이동합니다 `false` 인터넷 연결이 없거나 보관되지 않은 사이트의 경우 `true` 연결된 사이트의 경우. 의 설정 `true` 지원을 위해 익명 데이터를 NetApp에 전송할 수 있습니다. 기본 선택 옵션은 입니다 `false` 및 은 NetApp에 지원 데이터가 전송되지 않음을 나타냅니다. | 부울 | `false` (이 값은 기본값입니다.) |
| `autoSupport.url` | 둘 다 가능합니다 `enrolled` 또는 `url` 필드를 선택해야 합니다 | 이 URL은 익명 데이터를 보낼 위치를 결정합니다. | 문자열 | `https://support.netapp.com/asupprod/post/1.0/postAsup` |

이메일

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `email` | 를 변경합니다 `email` 문자열을 기본 초기 관리자 주소로 설정합니다. 에서 사용할 이 이메일 주소를 복사합니다 나중에. 이 이메일 주소는 UI에 로그인할 초기 계정의 사용자 이름으로 사용되며 Astra Control에서 이벤트를 알립니다. | 문자열 | `admin@example.com` |

이름

| 설정 | 지침 | 유형 | 예 |
| --- | --- | --- | --- |
| `firstName` | Astra 계정과 연결된 기본 초기 관리자의 이름입니다. 여기에 사용된 이름은 처음 로그인한 후 UI의 제목에 표시됩니다. | 문자열 | `SRE` |

성

| 설정 | 지침 | 유형 | 예 |
| --- | --- | --- | --- |
| `lastName` | Astra 계정과 연결된 기본 초기 관리자의 성. 여기에 사용된 이름은 처음 로그인한 후 UI의 제목에 표시됩니다. | 문자열 | `Admin` |

**imageRegistry(이미지 레지스트리)**

이 섹션에서 선택한 사항은 Astra 응용 프로그램 이미지, Astra Control Center Operator 및 Astra Control Center Helm 리포지토리를 호스팅하는 컨테이너 이미지 레지스트리를 정의합니다.

| 설정 | 사용 | 지침 | 유형 | 예 |
|---|---|---|---|---|
| `imageRegistry.name` | 필수 요소입니다 | Astra Control Center 배포에 필요한 모든 이미지를 호스팅하는 Astra Control 이미지 레지스트리의 이름입니다. 이 값은 미리 채워지며 로컬 레지스트리를 구성하지 않으면 아무 작업도 필요하지 않습니다. 로컬 레지스트리의 경우 이 기존 값을 에서 이미지를 푸시한 이미지 레지스트리 이름으로 바꿉니다 이전 단계. 사용하지 마십시오 `http://` 또는 `https://` 레지스트리 이름. | 문자열 | `cr.astra.netapp.io` (기본값) `example.registry.com/astra` (로컬 레지스트리 예) |

| 설정 | 사용 | 지침 | 유형 | 예 |
| --- | --- | --- | --- | --- |
| `imageRegistry.secret` | 선택 사항 | 이미지 레지스트리를 인증하는 데 사용되는 Kubernetes 비밀의 이름입니다. 값은 미리 채워지며 로컬 레지스트리와 에서 해당 레지스트리에 대해 입력한 문자열을 구성하지 않으면 아무 작업도 필요하지 않습니다 `imageRegistry.name` 비밀이 필요합니다.<br><br>중요: 인증이 필요하지 않은 로컬 레지스트리를 사용하는 경우 이를 삭제해야 합니다 `secret` 줄 내부 `imageRegistry` 그렇지 않으면 설치가 실패합니다. | 문자열 | `astra-registry-cred` |

**storageClass** 를 선택합니다

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `storageClass` | 를 변경합니다 `storageClass` 값 시작 `ontap-gold` 설치 시 필요한 다른 storageClass 리소스로 이동합니다. 명령을 실행합니다 `kubectl get sc` 구성된 기존 스토리지 클래스를 확인하려면 다음을 수행합니다. Astra Control Provisioner로 구성된 스토리지 클래스 중 하나를 매니페스트 파일에 입력해야 합니다 (`astra-control-center-<version>.manifest`) 및 는 Astra PVS에 사용됩니다. 이 옵션이 설정되어 있지 않으면 기본 스토리지 클래스가 사용됩니다.<br><br>참고: 기본 스토리지 클래스가 구성된 경우 기본 주석이 있는 유일한 스토리지 클래스인지 확인하십시오. | 문자열 | `ontap-gold` |

볼륨 리클레이임정책

| 설정 | 지침 | 유형 | 옵션 |
|---|---|---|---|
| `volumeReclaimPolicy` | 그러면 Astra의 PVS에 대한 재확보 정책이 설정됩니다. 이 정책을 으로 설정합니다 `Retain` Astra가 삭제된 후 영구 볼륨을 유지합니다. 이 정책을 으로 설정합니다 `Delete` Astra가 삭제된 후 영구 볼륨을 삭제합니다. 이 값을 설정하지 않으면 PVS가 유지됩니다. | 문자열 | • `Retain` (기본값)<br>• `Delete` |

응력 유형

| 설정 | 지침 | 유형 | 옵션 |
| --- | --- | --- | --- |
| `ingressType` | 다음 수신 유형 중 하나를 사용하십시오.<br><br>* 일반 * (`ingressType: "Generic"`) (기본값)<br>다른 수신 컨트롤러를 사용 중이거나 자체 수신 컨트롤러를 사용하려는 경우 이 옵션을 사용하십시오. Astra Control Center를 구축한 후 를 구성해야 합니다 "수신 컨트롤러" URL을 사용하여 Astra Control Center를 표시합니다.<br><br>중요: Astra Control Center에서 서비스 메시를 사용하려면 을 선택해야 합니다 `Generic` 수신 유형 으로 설정하고 직접 설정합니다 "수신 컨트롤러".<br><br>* AccTraefik * (`ingressType: "AccTraefik"`)<br>수신 컨트롤러를 구성하지 않으려는 경우 이 옵션을 사용하십시오. 그러면 Astra Control Center가 구축됩니다 `traefik` Kubernetes 로드 밸런서 유형 서비스로서의 게이트웨이<br><br>Astra Control Center는 "loadbalancer" 유형의 서비스를 사용합니다. (`svc/traefik` Astra Control Center 네임스페이스에서), 액세스 가능한 외부 IP 주소를 할당해야 합니다. 로드 밸런서가 사용자 환경에서 허용되고 아직 로드 밸런서가 구성되어 있지 않은 경우 MetalLB 또는 다른 외부 서비스 로드 밸런서를 사용하여 외부 IP 주소를 서비스에 할당할 수 있습니다. 내부 | 문자열 | • `Generic` (기본값)<br>• `AccTraefik` |

스케일크기

| 설정 | 지침 | 유형 | 옵션 |
|---|---|---|---|
| `scaleSize` | 기본적으로 Astra는 HA(High Availability)를 사용합니다. `scaleSize` 의 `Medium`즉, `HA`에서 대부분의 서비스를 구축하고 이중화를 위해 여러 복제본을 배포합니다. 와 함께 `scaleSize` 현재 `Small`Astra는 소비를 줄이기 위한 필수 서비스를 제외한 모든 서비스의 복제본 수를 줄일 것입니다. 팁: `Medium` 약 100개의 Pod로 구축 가능(임시 워크로드 제외) 100 Pod는 3개의 마스터 노드 및 3개의 작업자 노드 구성을 기반으로 합니다.) 특히 재해 복구 시나리오를 고려할 때 사용자 환경에서 문제가 될 수 있는 Pod별 네트워크 제한 사항에 유의하십시오. | 문자열 | • `Small`<br>• `Medium` (기본값) |

**astraResourcaresaTM**

| 설정 | 지침 | 유형 | 옵션 |
|---|---|---|---|
| `astraResourcesScaler` | AstraControlCenter 리소스 제한에 대한 확장 옵션 기본적으로 Astra Control Center는 Astra 내의 대부분의 구성 요소에 대해 설정된 리소스 요청과 함께 배포됩니다. 이 구성을 통해 Astra Control Center 소프트웨어 스택은 애플리케이션 로드 및 확장 수준이 높은 환경에서 더 나은 성능을 발휘할 수 있습니다. 그러나 더 작은 개발 또는 테스트 클러스터를 사용하는 시나리오에서는 CR 필드를 사용합니다 `astraResourcesScalar` 로 설정할 수 있습니다 `Off`. 이렇게 하면 리소스 요청이 비활성화되고 소규모 클러스터에 구축할 수 있습니다. | 문자열 | • `Default` (기본값)<br>• `Off` |

추가 가치입니다

설치 시 알려진 문제를 방지하려면 Astra Control Center CR에 다음 추가 값을 추가합니다.

```
additionalValues:
    keycloak-operator:
      livenessProbe:
        initialDelaySeconds: 180
      readinessProbe:
        initialDelaySeconds: 180
```

**CRD**

이 섹션에서 선택한 사항은 Astra Control Center에서 CRD를 처리하는 방법을 결정합니다.

| 설정 | 지침 | 유형 | 예 |
|---|---|---|---|
| `crds.externalCertManager` | 외부 인증서 관리자를 사용하는 경우 를 변경합니다 `externalCertManager` 를 선택합니다 `true`. 기본값입니다 `false` 설치 중에 Astra Control Center가 자체 인증서 관리자 CRD를 설치합니다. CRD는 클러스터 전체 오브젝트이며 이를 설치하면 클러스터의 다른 부분에 영향을 줄 수 있습니다. 이 플래그를 사용하여 Astra Control Center에 이러한 CRD가 Astra Control Center 외부의 클러스터 관리자에 의해 설치 및 관리된다는 신호를 보낼 수 있습니다. | 부울 | `False` (이 값은 기본값입니다.) |
| `crds.externalTraefik` | 기본적으로 Astra Control Center는 필요한 Traefik CRD를 설치합니다. CRD는 클러스터 전체 오브젝트이며 이를 설치하면 클러스터의 다른 부분에 영향을 줄 수 있습니다. 이 플래그를 사용하여 Astra Control Center에 이러한 CRD가 Astra Control Center 외부의 클러스터 관리자에 의해 설치 및 관리된다는 신호를 보낼 수 있습니다. | 부울 | `False` (이 값은 기본값입니다.) |

설치를 완료하기 전에 구성에 맞는 올바른 스토리지 클래스 및 수신 유형을 선택했는지 확인하십시오.

**Astra_CONTROL_CENTER.YAML**을 샘플링합니다

```
apiVersion: astra.netapp.io/v1
kind: AstraControlCenter
metadata:
  name: astra
spec:
  accountName: "Example"
  astraVersion: "ASTRA_VERSION"
  astraAddress: "astra.example.com"
  autoSupport:
    enrolled: true
  email: "[admin@example.com]"
  firstName: "SRE"
  lastName: "Admin"
  imageRegistry:
    name: "[cr.astra.netapp.io or your_registry_path]"
    secret: "astra-registry-cred"
  storageClass: "ontap-gold"
  volumeReclaimPolicy: "Retain"
  ingressType: "Generic"
  scaleSize: "Medium"
  astraResourcesScaler: "Default"
  additionalValues:
    keycloak-operator:
      livenessProbe:
        initialDelaySeconds: 180
      readinessProbe:
        initialDelaySeconds: 180
  crds:
    externalTraefik: false
    externalCertManager: false
```

# Astra 제어 센터 및 운전자 설치를 완료합니다

1. 이전 단계에서 작성하지 않은 경우, "NetApp-acc"(또는 사용자 지정) 네임스페이스를 작성하십시오.

```
kubectl create ns [netapp-acc or custom namespace]
```

2. Astra Control Center에서 서비스 메시를 사용하는 경우 에 다음 레이블을 추가합니다 `netapp-acc` 또는 사용자 지정 네임스페이스:

> 수신 유형입니다 (`ingressType`)를 로 설정해야 합니다 `Generic` Astra Control Center CR에서 이 명령을 진행하기 전에

```
kubectl label ns [netapp-acc or custom namespace] istio-
injection:enabled
```

3. (권장) "엄격한 MTL을 활성화합니다" Istio 서비스 메시의 경우:

```
kubectl apply -n istio-system -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT
EOF
```

4. "NetApp-acc"(또는 사용자 지정) 네임스페이스에 Astra Control Center를 설치합니다.

```
kubectl apply -f astra_control_center.yaml -n [netapp-acc or custom
namespace]
```

> Astra Control Center 운영자는 환경 요구 사항에 대한 자동 검사를 실행합니다. 없습니다 "요구 사항" 설치가 실패하거나 Astra Control Center가 제대로 작동하지 않을 수 있습니다. 를 참조하십시오 다음 섹션을 참조하십시오 자동 시스템 점검과 관련된 경고 메시지를 확인합니다.

# 시스템 상태를 확인합니다

kubeck 명령을 사용하여 시스템 상태를 확인할 수 있습니다. OpenShift를 사용하려는 경우 검증 단계에 유사한 OC 명령을 사용할 수 있습니다.

단계

1. 설치 프로세스에서 유효성 검사와 관련된 경고 메시지가 생성되지 않았는지 확인합니다.

```
kubectl get acc [astra or custom Astra Control Center CR name] -n
[netapp-acc or custom namespace] -o yaml
```

> Astra Control Center 운영자 로그에도 추가 경고 메시지가 표시됩니다.

2. 자동화된 요구 사항 확인을 통해 보고된 환경 관련 문제를 모두 해결하십시오.

> 사용자 환경이 을(를) 충족하는지 확인하여 문제를 해결할 수 있습니다 "요구 사항" Astra Control Center의 경우

3. 모든 시스템 구성 요소가 성공적으로 설치되었는지 확인합니다.

```
kubectl get pods -n [netapp-acc or custom namespace]
```

각 포드는 'Running' 상태여야 합니다. 시스템 포드를 구축하는 데 몇 분 정도 걸릴 수 있습니다.

샘플 응답을 위해 확장합니다

```
acc-helm-repo-5bd77c9ddd-8wxm2             1/1     Running     0
1h
activity-5bb474dc67-8l9ss                  1/1     Running     0
1h
activity-5bb474dc67-qbrtq                  1/1     Running     0
1h
api-token-authentication-6wbj2             1/1     Running     0
1h
api-token-authentication-9pgw6             1/1     Running     0
1h
api-token-authentication-tqf6d             1/1     Running     0
1h
asup-5495f44dbd-z4kft                      1/1     Running     0
1h
authentication-6fdd899858-5x45s            1/1     Running     0
1h
bucketservice-84d47487d-n9xgp              1/1     Running     0
1h
bucketservice-84d47487d-t5jhm              1/1     Running     0
1h
cert-manager-5dcb7648c4-hbldc              1/1     Running     0
1h
cert-manager-5dcb7648c4-nr9qf              1/1     Running     0
1h
cert-manager-cainjector-59b666fb75-bk2tf   1/1     Running     0
1h
cert-manager-cainjector-59b666fb75-pfnck   1/1     Running     0
1h
cert-manager-webhook-c6f9b6796-ngz2x       1/1     Running     0
1h
cert-manager-webhook-c6f9b6796-rwtbn       1/1     Running     0
1h
certificates-5f5b7b4dd-52tnj               1/1     Running     0
1h
certificates-5f5b7b4dd-gtjbx               1/1     Running     0
1h
certificates-expiry-check-28477260-dz5vw   0/1     Completed   0
1h
cloud-extension-6f58cc579c-lzfmv           1/1     Running     0
1h
cloud-extension-6f58cc579c-zw2km           1/1     Running     0
1h
cluster-orchestrator-79dd5c8d95-qjg92      1/1     Running     0
```

```
1h
composite-compute-85dc84579c-nz82f         1/1     Running     0
1h
composite-compute-85dc84579c-wx2z2         1/1     Running     0
1h
composite-volume-bff6f4f76-789nj           1/1     Running     0
1h
composite-volume-bff6f4f76-kwnd4           1/1     Running     0
1h
credentials-79fd64f788-m7m8f               1/1     Running     0
1h
credentials-79fd64f788-qnc6c               1/1     Running     0
1h
entitlement-f69cdbd77-4p2kn                1/1     Running     0
1h
entitlement-f69cdbd77-hswm6                1/1     Running     0
1h
features-7b9585444c-7xd7m                  1/1     Running     0
1h
features-7b9585444c-dcqwc                  1/1     Running     0
1h
fluent-bit-ds-crq8m                        1/1     Running     0
1h
fluent-bit-ds-gmgq8                        1/1     Running     0
1h
fluent-bit-ds-gzr4f                        1/1     Running     0
1h
fluent-bit-ds-j6sf6                        1/1     Running     0
1h
fluent-bit-ds-v4t9f                        1/1     Running     0
1h
fluent-bit-ds-x7j59                        1/1     Running     0
1h
graphql-server-6cc684fb46-2x8lr            1/1     Running     0
1h
graphql-server-6cc684fb46-bshbd            1/1     Running     0
1h
hybridauth-84599f79fd-fjc7k                1/1     Running     0
1h
hybridauth-84599f79fd-s9pmn                1/1     Running     0
1h
identity-95df98cb5-dvlmz                   1/1     Running     0
1h
identity-95df98cb5-krf59                   1/1     Running     0
1h
influxdb2-0                                1/1     Running     0
```

```
1h
keycloak-operator-6d4d688697-cfq8b         1/1     Running     0
1h
krakend-5d5c8f4668-7bq8g                   1/1     Running     0
1h
krakend-5d5c8f4668-t8hbn                   1/1     Running     0
1h
license-689cdd4595-2gsc8                   1/1     Running     0
1h
license-689cdd4595-g6vwk                   1/1     Running     0
1h
login-ui-57bb599956-4fwgz                  1/1     Running     0
1h
login-ui-57bb599956-rhztb                  1/1     Running     0
1h
loki-0                                     1/1     Running     0
1h
metrics-facade-846999bdd4-f7jdm            1/1     Running     0
1h
metrics-facade-846999bdd4-lnsxl            1/1     Running     0
1h
monitoring-operator-6c9d6c4b8c-ggkrl       2/2     Running     0
1h
nats-0                                     1/1     Running     0
1h
nats-1                                     1/1     Running     0
1h
nats-2                                     1/1     Running     0
1h
natssync-server-6df7d6cc68-9v2gd           1/1     Running     0
1h
nautilus-64b7fbdd98-bsgwb                  1/1     Running     0
1h
nautilus-64b7fbdd98-djlhw                  1/1     Running     0
1h
openapi-864584bccc-75nlv                   1/1     Running     0
1h
openapi-864584bccc-zh6bx                   1/1     Running     0
1h
polaris-consul-consul-server-0             1/1     Running     0
1h
polaris-consul-consul-server-1             1/1     Running     0
1h
polaris-consul-consul-server-2             1/1     Running     0
1h
polaris-keycloak-0                         1/1     Running     2 (1h
```

```
ago)      1h
polaris-keycloak-1                          1/1     Running     0
1h
polaris-keycloak-db-0                       1/1     Running     0
1h
polaris-keycloak-db-1                       1/1     Running     0
1h
polaris-keycloak-db-2                       1/1     Running     0
1h
polaris-mongodb-0                           1/1     Running     0
1h
polaris-mongodb-1                           1/1     Running     0
1h
polaris-mongodb-2                           1/1     Running     0
1h
polaris-ui-66476dcf87-f6s8j                 1/1     Running     0
1h
polaris-ui-66476dcf87-ztjk7                 1/1     Running     0
1h
polaris-vault-0                             1/1     Running     0
1h
polaris-vault-1                             1/1     Running     0
1h
polaris-vault-2                             1/1     Running     0
1h
public-metrics-bfc4fc964-x4m79              1/1     Running     0
1h
storage-backend-metrics-7dbb88d4bc-g78cj    1/1     Running     0
1h
storage-provider-5969b5df5-hjvcm            1/1     Running     0
1h
storage-provider-5969b5df5-r79ld            1/1     Running     0
1h
task-service-5fc9dc8d99-4q4f4               1/1     Running     0
1h
task-service-5fc9dc8d99-8l5zl               1/1     Running     0
1h
task-service-task-purge-28485735-fdzkd      1/1     Running     0
12m
telegraf-ds-2rgm4                           1/1     Running     0
1h
telegraf-ds-4qp6r                           1/1     Running     0
1h
telegraf-ds-77frs                           1/1     Running     0
1h
telegraf-ds-bc725                           1/1     Running     0
```

```
1h
telegraf-ds-cvmxf                          1/1     Running     0
1h
telegraf-ds-tqzgj                          1/1     Running     0
1h
telegraf-rs-5wtd8                          1/1     Running     0
1h
telemetry-service-6747866474-5djnc         1/1     Running     0
1h
telemetry-service-6747866474-thb7r         1/1     Running     1 (1h
ago)      1h
tenancy-5669854fb6-gzdzf                   1/1     Running     0
1h
tenancy-5669854fb6-xvsm2                   1/1     Running     0
1h
traefik-8f55f7d5d-4lgfw                    1/1     Running     0
1h
traefik-8f55f7d5d-j4wt6                    1/1     Running     0
1h
traefik-8f55f7d5d-p6gcq                    1/1     Running     0
1h
trident-svc-7cb5bb4685-54cnq               1/1     Running     0
1h
trident-svc-7cb5bb4685-b28xh               1/1     Running     0
1h
vault-controller-777b9bbf88-b5bqt          1/1     Running     0
1h
vault-controller-777b9bbf88-fdfd8          1/1     Running     0
1h
```

4. (선택 사항) 을(를) 확인합니다 `acc-operator` 진행 상황을 모니터링하기 위한 로그:

```
kubectl logs deploy/acc-operator-controller-manager -n netapp-acc-
operator -c manager -f
```

> `accHost` 클러스터 등록은 마지막 작업 중 하나이며, 클러스터 등록에 실패하면 배포에 실패하지 않습니다. 로그에 클러스터 등록 실패가 표시되는 경우 를 통해 등록을 다시 시도할 수 있습니다 "UI에서 클러스터 워크플로우를 추가합니다" API를 사용합니다.

5. 모든 Pod가 실행되면 설치가 성공적으로 완료되었는지 확인합니다 (`READY` 있습니다 `True`) 및 Astra Control Center에 로그인할 때 사용할 초기 설정 암호를 받습니다.

```
kubectl get AstraControlCenter -n [netapp-acc or custom namespace]
```

응답:

```
NAME    UUID                                   VERSION     ADDRESS
READY
astra   9aa5fdae-4214-4cb7-9976-5d8b4c0ce27f  24.02.0-69
10.111.111.111  True
```

> UUID 값을 복사합니다. 암호는 ACC-, UUID 값( ACC-[UUID]), 이 예에서는 ACC-9aa5faaaaaaaud-4214-4cb7-9976-5d8b4c0ce27f)입니다.

# 부하 분산을 위한 수신 설정

서비스에 대한 외부 액세스를 관리하는 Kubernetes 수신 컨트롤러를 설정할 수 있습니다. 이 절차에서는 기본값을 사용한 경우 수신 컨트롤러에 대한 설정 예제를 제공합니다 `ingressType: "Generic"` Astra Control Center 사용자 지정 리소스 (`astra_control_center.yaml`)를 클릭합니다. 지정한 경우 이 절차를 사용할 필요가 없습니다 `ingressType: "AccTraefik"` Astra Control Center 사용자 지정 리소스 (`astra_control_center.yaml`)를 클릭합니다.

Astra Control Center가 구축된 후 Astra Control Center가 URL로 노출되도록 수신 컨트롤러를 구성해야 합니다.

설치 단계는 사용하는 수신 컨트롤러의 유형에 따라 다릅니다. Astra Control Center는 다양한 수신 컨트롤러 유형을 지원합니다. 이러한 설정 절차는 일반적인 수신 컨트롤러 유형의 예를 제공합니다.

시작하기 전에

- 필수 요소입니다 "수신 컨트롤러" 이미 배포되어 있어야 합니다.
- 를 클릭합니다 "수신 클래스" 수신 컨트롤러에 해당하는 컨트롤러가 이미 생성되어야 합니다.

## Istio 침투에 대한 단계

1. Istio Ingress를 구성합니다.

   > 이 절차에서는 "기본" 구성 프로파일을 사용하여 Istio를 구축한다고 가정합니다.

2. 수신 게이트웨이에 대해 원하는 인증서 및 개인 키 파일을 수집하거나 생성합니다.

   CA 서명 또는 자체 서명 인증서를 사용할 수 있습니다. 공통 이름은 Astra 주소(FQDN)여야 합니다.

   명령 예:

   ```
   openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out
   tls.crt
   ```

3. 암호를 만듭니다 `tls secret name` 유형 `kubernetes.io/tls` 에서 TLS 개인 키 및 인증서의 경우 `istio-system namespace` TLS 비밀에 설명되어 있습니다.

명령 예:

```
kubectl create secret tls [tls secret name] --key="tls.key"
--cert="tls.crt" -n istio-system
```

비밀의 이름은 'istio-ingress.YAML' 파일에 제공된 'pec.tls.secretName'과 일치해야 합니다.

4. 에 수신 리소스를 배포합니다 `netapp-acc` (또는 사용자 지정 이름) 스키마에 대해 v1 리소스 형식을 사용하는 네임스페이스입니다 (`istio-Ingress.yaml` 이 예에서 사용됨):

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: istio
spec:
  controller: istio.io/ingress-controller
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress
  namespace: [netapp-acc or custom namespace]
spec:
  ingressClassName: istio
  tls:
  - hosts:
    - <ACC address>
    secretName: [tls secret name]
  rules:
  - host: [ACC address]
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: traefik
            port:
              number: 80
```

5. 변경 사항 적용:

```
kubectl apply -f istio-Ingress.yaml
```

6. 수신 상태를 점검하십시오.

```
kubectl get ingress -n [netapp-acc or custom namespace]
```

응답:

```
NAME    CLASS HOSTS              ADDRESS         PORTS   AGE
ingress istio astra.example.com 172.16.103.248  80, 443 1h
```

7. Astra Control Center 설치를 완료합니다.

## Nginx 수신 컨트롤러 단계

1. 형식의 암호를 만듭니다 `kubernetes.io/tls` 에서 TLS 개인 키 및 인증서의 경우 `netapp-acc` 에 설명된 대로 (또는 사용자 지정 이름) 네임스페이스를 사용합니다 "TLS 비밀".
2. 수신 리소스를 에 배포합니다 `netapp-acc` (또는 사용자 지정 이름) 스키마에 대해 v1 리소스 형식을 사용하는 네임스페이스입니다 (`nginx-Ingress.yaml` 이 예에서 사용됨):

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: netapp-acc-ingress
  namespace: [netapp-acc or custom namespace]
spec:
  ingressClassName: [class name for nginx controller]
  tls:
  - hosts:
    - <ACC address>
    secretName: [tls secret name]
  rules:
  - host: <ACC address>
    http:
      paths:
        - path:
          backend:
            service:
              name: traefik
              port:
                number: 80
          pathType: ImplementationSpecific
```

3. 변경 사항 적용:

```
kubectl apply -f nginx-Ingress.yaml
```

Nginx 컨트롤러를 이 아닌 배포로 설치하는 것이 좋습니다 `daemonSet`.

### OpenShift Ingress 컨트롤러를 위한 단계

1. 인증서를 구입하고 OpenShift 라우트에서 사용할 수 있도록 준비된 키, 인증서 및 CA 파일을 가져옵니다.
2. OpenShift 경로를 생성합니다.

```
oc create route edge --service=traefik --port=web -n [netapp-acc or
custom namespace] --insecure-policy=Redirect --hostname=<ACC address>
--cert=cert.pem --key=key.pem
```

## Astra Control Center UI에 로그인합니다

Astra Control Center를 설치한 후 기본 관리자의 암호를 변경하고 Astra Control Center UI 대시보드에 로그인합니다.

단계

1. 브라우저에서 FQDN(을 포함)을 입력합니다 `https://` 접두사)를 입력합니다 `astraAddress` 에 있습니다 `astra_control_center.yaml` CR Astra Control Center를 설치했습니다.
2. 메시지가 표시되면 자체 서명된 인증서를 수락합니다.

   로그인 후 사용자 지정 인증서를 만들 수 있습니다.

3. Astra Control Center 로그인 페이지에서 에 사용한 값을 입력합니다 `email` 인치 `astra_control_center.yaml` CR Astra Control Center를 설치했습니다를 누른 다음 초기 설치 암호를 입력합니다 (`ACC-[UUID]`)를 클릭합니다.

   잘못된 암호를 세 번 입력하면 15분 동안 관리자 계정이 잠깁니다.

4. Login * 을 선택합니다.
5. 메시지가 나타나면 암호를 변경합니다.

   첫 번째 로그인인 경우 암호를 잊어버리고 다른 관리 사용자 계정이 아직 생성되지 않은 경우 에 문의하십시오 "NetApp 지원" 비밀번호 복구 지원을 위해.

6. (선택 사항) 기존의 자체 서명된 TLS 인증서를 제거하고 로 바꿉니다 "인증 기관(CA)에서 서명한 사용자 지정 TLS 인증서".

# 설치 문제를 해결합니다

서비스 중 '오류' 상태인 서비스가 있으면 로그를 검사할 수 있습니다. 400 ~ 500 범위의 API 응답 코드를 찾습니다. 이는 고장이 발생한 장소를 나타냅니다.

옵션

- Astra Control Center 운영자 로그를 검사하려면 다음을 입력하십시오.

```
kubectl logs deploy/acc-operator-controller-manager -n netapp-acc-
operator -c manager -f
```

- Astra Control Center CR의 출력을 확인하려면:

```
kubectl get acc -n [netapp-acc or custom namespace] -o yaml
```

# 대체 설치 절차

- * Red Hat OpenShift OperatorHub를 사용하여 설치 *: 사용 "대체 절차" OperatorHub를 사용하여 OpenShift에 Astra Control Center를 설치하려면
- * Cloud Volumes ONTAP 백엔드를 사용하여 퍼블릭 클라우드에 설치 *: 사용 "수행할 수 있습니다" AWS(Amazon Web Services), GCP(Google Cloud Platform) 또는 Cloud Volumes ONTAP 스토리지 백엔드가 있는 Microsoft Azure에 Astra Control Center를 설치하려면 다음을 수행합니다.

# 다음 단계

- (선택 사항) 환경에 따라 사후 설치를 완료합니다 "구성 단계".
- "Astra Control Center를 설치하고 UI에 로그인하여 암호를 변경하면 라이센스를 설정하고, 클러스터를 추가하고, 인증을 활성화하고, 스토리지를 관리하고, 버킷을 추가할 수 있습니다".

# 외부 인증서 관리자를 구성합니다

## Kubernetes 클러스터에 이미 인증 관리자가 있는 경우, Astra Control Center에서 자체 인증 관리자를 설치하지 않도록 몇 가지 필수 단계를 수행해야 합니다.

단계

1. 인증서 관리자가 설치되었는지 확인합니다.

```
kubectl get pods -A | grep 'cert-manager'
```

샘플 반응:

```
cert-manager   essential-cert-manager-84446f49d5-sf2zd              1/1
Running    0      6d5h
cert-manager   essential-cert-manager-cainjector-66dc99cc56-9ldmt   1/1
Running    0      6d5h
cert-manager   essential-cert-manager-webhook-56b76db9cc-fjqrq      1/1
Running    0      6d5h
```

2. 에 대한 인증서/키 쌍을 생성합니다 `astraAddress` FQDN:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out
tls.crt
```

샘플 반응:

```
Generating a 2048 bit RSA private key
..................+++
.........................................+++
writing new private key to 'tls.key'
```

3. 이전에 생성된 파일을 사용하여 암호 생성:

```
kubectl create secret tls selfsigned-tls --key tls.key --cert tls.crt -n
<cert-manager-namespace>
```

샘플 반응:

```
secret/selfsigned-tls created
```

4. 을 생성합니다 `ClusterIssuer` 정확히 * 인 파일 다음은 같지만 가 있는 네임스페이스 위치가 포함되어 있습니다 `cert-manager` Pod가 설치된 경우:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: astra-ca-clusterissuer
  namespace: <cert-manager-namespace>
spec:
  ca:
    secretName: selfsigned-tls
```

```
kubectl apply -f ClusterIssuer.yaml
```

샘플 반응:

```
clusterissuer.cert-manager.io/astra-ca-clusterissuer created
```

5. 를 확인합니다 `ClusterIssuer` 이(가) 올바르게 나타납니다. `Ready` 은(는) 이어야 합니다 `True` 계속 진행하려면:

```
kubectl get ClusterIssuer
```

샘플 반응:

```
NAME                      READY   AGE
astra-ca-clusterissuer    True    9s
```

6. 를 완료합니다 "Astra Control Center 설치 프로세스". A가 있습니다 "Astra Control Center 클러스터 YAML에 필요한 구성 단계" 인증서 관리자가 외부에 설치되었음을 나타내기 위해 CRD 값을 변경합니다. Astra Control Center가 외부 인증서 관리자를 인식하도록 설치 중에 이 단계를 완료해야 합니다.

저작권 정보

Copyright © 2024 NetApp, Inc. All Rights Reserved. 미국에서 인쇄된 본 문서의 어떠한 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 수단(복사, 녹음, 녹화 또는 전자 검색 시스템에 저장하는 것을 비롯한 그래픽, 전자적 또는 기계적 방법)으로도 복제될 수 없습니다.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 '있는 그대로' 제공되며 상품성 및 특정 목적에의 적합성에 대한 명시적 또는 묵시적 보증을 포함하여(이에 제한되지 않음) 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 영업 중단을 포함하여(이에 국한되지 않음), 이 소프트웨어의 사용으로 인해 발생하는 모든 직접 및 간접 손해, 우발적 손해, 특별 손해, 징벌적 손해, 결과적 손해의 발생에 대하여 그 발생 이유, 책임론, 계약 여부, 엄격한 책임, 불법 행위(과실 또는 그렇지 않은 경우)와 관계없이 어떠한 책임도 지지 않으며, 이와 같은 손실의 발생 가능성이 통지되었다 하더라도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구매의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허, 해외 특허 또는 출원 중인 특허로 보호됩니다.

제한적 권리 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.227-7013(2014년 2월) 및 FAR 52.227-19(2007년 12월)의 기술 데이터-비상업적 품목에 대한 권리(Rights in Technical Data -Noncommercial Items) 조항의 하위 조항 (b)(3)에 설명된 제한사항이 적용됩니다.

여기에 포함된 데이터는 상업용 제품 및/또는 상업용 서비스(FAR 2.101에 정의)에 해당하며 NetApp, Inc.의 독점 자산입니다. 본 계약에 따라 제공되는 모든 NetApp 기술 데이터 및 컴퓨터 소프트웨어는 본질적으로 상업용이며 개인 비용만으로 개발되었습니다. 미국 정부는 데이터가 제공된 미국 계약과 관련하여 해당 계약을 지원하는 데에만 데이터에 대한 전 세계적으로 비독점적이고 양도할 수 없으며 재사용이 불가능하며 취소 불가능한 라이센스를 제한적으로 가집니다. 여기에 제공된 경우를 제외하고 NetApp, Inc.의 사전 서면 승인 없이는 이 데이터를 사용, 공개, 재생산, 수정, 수행 또는 표시할 수 없습니다. 미국 국방부에 대한 정부 라이센스는 DFARS 조항 252.227-7015(b)(2014년 2월)에 명시된 권한으로 제한됩니다.

상표 정보

NETAPP, NETAPP 로고 및 http://www.netapp.com/TM에 나열된 마크는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.